An Ordering Doppelgänger

ML for relationship-driven distribution. A production-grade system for defending share where every SKU, on every order cycle, is a battle for the next sale. Built as a personal project, informed by deep domain experience in B2B distribution sales and analytics. Architected for per-distributor deployment.

7,080	53	19	72.2%
lines of Python	engineered features	Postgres tables	AUC-PR · 681K samples

The problem

Relationship-driven distribution (foodservice, industrial supply, electrical and HVAC, medical and dental, building products, automotive parts) is one of the most competitive corners of B2B in the United States. Customers buy from multiple distributors simultaneously, often with no purchase minimums, no exclusivity, and no switching cost. Competing distributors in relationship-driven categories carry largely overlapping SKU coverage, which means a customer's product preferences don't lock them to any one vendor. Competitors can fill the same demand from nearly identical product catalogs.

What earns share, and defends it, is the **relationship**: account executive (AE) responsiveness, fill-rate reliability, and making the customer's life easier during the hours when their time is most expensive. Buyers in these industries are time-starved (running shifts, managing technicians, handling patient flow), and asking them to scroll through hundreds of SKUs is administrative work at exactly the wrong moment.

Despite this dynamic, most reorder workflows are still manual: the AE calls or emails, the customer scrolls through hundreds of SKUs, and nobody knows which items the customer just bought from a competitor until the case-count report comes in two weeks later. Lost opportunities pile up: missed substitutions on out-of-stock items, forgotten staples picked up elsewhere, and a slow erosion of account penetration that nobody catches until the customer is gone. Years working in distribution sales and analytics make the structural gap obvious: the data to detect this erosion already sits in every distributor's ERP. The system to act on it does not.

What the system does

A predictive cart is a service-quality tool inside that relationship dynamic. On each customer's regular order cycle, the engine generates a pre-filled order. Items the customer is most likely to need this period, at the quantities they typically order, with substitutes already mapped for anything out of stock. The customer confirms in one click during whatever short window they have between operational fires; one click emits an EDI 850 to the distributor's existing pipeline. Each cart is paired with a streaming natural-language explanation generated by Claude and fact-checked by a second Claude pass. This critic-validator pattern prevents hallucinated quantities in customer-facing copy. The AE spends face time on partnership and problem-solving rather than rote order-taking that software can handle.

The system gets meaningfully better over time. After roughly six months of usage, the cart becomes what I think of as an **ordering doppelgänger**: a calibrated model of how this specific customer actually orders, tuned to their cycle rhythms, seasonal swings, substitution patterns, and personal acceptance behavior. The early carts are useful; the mature carts are uncanny.

Underneath the service-quality UX, the model is doing something more pointed: it's surfacing **account vulnerability**. A high-confidence prediction the customer doesn't accept isn't a model error. It's a signal the customer's behavior diverged from their doppelgänger. Maybe a competitor moved on a slot; maybe the customer's operations changed; maybe the AE

hasn't been in touch in two weeks. The dismissal becomes a conversation starter for the AE. Input into the relationship, not a replacement for it.

Engineering principle

In a relationship-driven industry, software lives or dies by whether the AE trusts it. An ignored alert is worse than no alert, because it conditions the AE to filter out the signal. Every prediction is paired with a human-readable explanation, every threshold is tunable, and the system runs in silent learning mode for 4–6 weeks before any customer-facing notification goes live. The goal isn't model accuracy in isolation. It's deployable service quality and AE credibility, because in relationship-driven distribution the AE *is* the relationship, and the system exists to make that relationship easier to maintain, not to replace it.

System surface

Layer	Components	Stack
Data	19-table Postgres schema, UUID keys, conflict-resolving upserts	PostgreSQL 17 · Supabase · RLS
ML Pipeline	Feature engineering, STL seasonality, LightGBM training, health monitoring	Python · pandas · statsmodels · LightGBM
Inference	Trigger engine, cart API, webhooks, substitution engine	Python · FastAPI · SQLAlchemy
Integration	ERP order-guide sync, EDI 850/846/832 bridge	X12 EDI · SPS · TrueCommerce · GXS
Web	Admin console, customer portal, AI cart summary, insight + alerts	Next.js 16 · React 19 · shadcn/ui · Anthropic SDK
Hosting	Frontend on Vercel · Backend on Fly.io (IPv6-capable for Supabase)	Vercel · Fly.io · Supabase Auth (PKCE)
Ops	Model registry, accuracy tracker, threshold optimizer, silent mode	Sentry · Playwright · Lighthouse CI · Percy

On benchmarking: why the wrong metric is dangerous

Reorder prediction in relationship-driven distribution is meaningfully harder than the recommendation problems most ML benchmarks are tuned for. There's no clickstream, no time-on-page, no add-to-cart sequencing. Only historical order data, seasonality, and the constant headwind of competing distributors invisibly poaching SKUs from accounts. Published benchmarks in adjacent cold-data reorder problems typically land in the 55–65% precision range; the operational threshold for usable cart pre-fill is around 60%, because below that the AE override rate becomes a productivity drag and the AE starts ignoring the model.

The system clears that bar: **AUC-PR 72.2%, precision 63.1%, recall 68.5%, F1 66.7%** validated on 681,638 samples. The quantity predictor lands at **MAE 0.26 units** on 82,631 validation samples. But raw precision is the wrong primary metric. The model's real job is to deliver *service quality*: easier ordering for the customer, more efficient face time for the AE, and earlier visibility into accounts where behavior is drifting from the established pattern. The threshold optimizer is tuned against operational outcomes (acceptance rate, override rate, OOS-substitute recovery, case-count trend) rather than F1 in isolation, because the system is graded on *customer ease and AE credibility*, with share-defense as the downstream outcome, not on confusion-matrix entries.

What you'll find in this document

Page 3 covers the ML architecture: two-model decomposition, the case for LightGBM, and the hyperparameters. **Page 4** walks through 53 engineered features, the gain analysis, and the structural insight about why DOW-specific signals dominate in relationship-driven distribution. **Page 5** goes deep on STL seasonality and the four-level fallback. **Page 6** covers training discipline and the operational maturity built specifically to protect AE credibility: silent learning mode, threshold optimizer, behavioral drift monitoring. **Page 7** covers the three Claude-powered surfaces and the critic-validator pattern. **Page 8** covers engineering decisions worth noting and the deployment stack. **Page 9** is the roadmap, including productized divergence scoring, plus honest framing and bio. **Page 10** is a full-page system architecture diagram.

ML architecture: two models, 53 features

Why a two-model decomposition, why LightGBM, and the hyperparameters that converged.

Two-model decomposition

The naive approach to reorder prediction is a single multi-output model emitting (probability, quantity) per item. I chose a two-model decomposition instead: a binary basket predictor that answers *will this item be ordered this week?*, and a quantity regressor that answers *if yes, how much?*, for three reasons:

- **Loss functions don't agree.** Binary cross-entropy and quantity MAE optimize fundamentally different gradients. Joint training forces a compromise that hurts both.
- **Class imbalance is severe.** In any given customer-week, ~12% of guide items are positives. The quantity model should never see the negatives at all; the basket model needs `scale_pos_weight` to handle them.
- **Inference logic is cleaner.** The trigger engine applies tunable thresholds (default 0.60) on the basket model independently from quantity rounding, which simplifies the threshold optimizer.

LightGBM, specifically

Both models use LightGBM. The choice was deliberate over XGBoost, scikit-learn ensembles, and a neural baseline:

- **Categorical handling.** Customer regions, product categories, and warehouses are all natively categorical. LightGBM's categorical splits avoid the one-hot explosion that hurts both training speed and memory.
- **Speed and footprint.** Full training completes in ~15 minutes on a single machine. Retraining weekly is operationally cheap.
- **Interpretable feature gain.** Gain-based feature importance is meaningful for tree ensembles in a way SHAP-on-deep-models is not. Operators can look at the top features and understand model behavior.
- **L1 regression objective for quantity.** Distribution quantities are fat-tailed. The occasional large order at 10× the customer's usual is real signal, not noise. MAE loss handles those rows without letting them dominate the gradient.

Hyperparameters worth noting

Basket model: `learning_rate=0.05, num_leaves=63, min_child_samples=30, min_gain_to_split=0.01, lambda_l1=0.1, feature_fraction=0.8, bagging_fraction=0.8, bagging_freq=5`. Early stopping on binary_logloss with 50-round patience converged at 18 trees on the current dataset. The model stopped finding signal quickly, which suggests the feature set is rich and the regularization is right. The quantity model uses identical settings except `objective=regression_l1`, and converged at 362 trees.

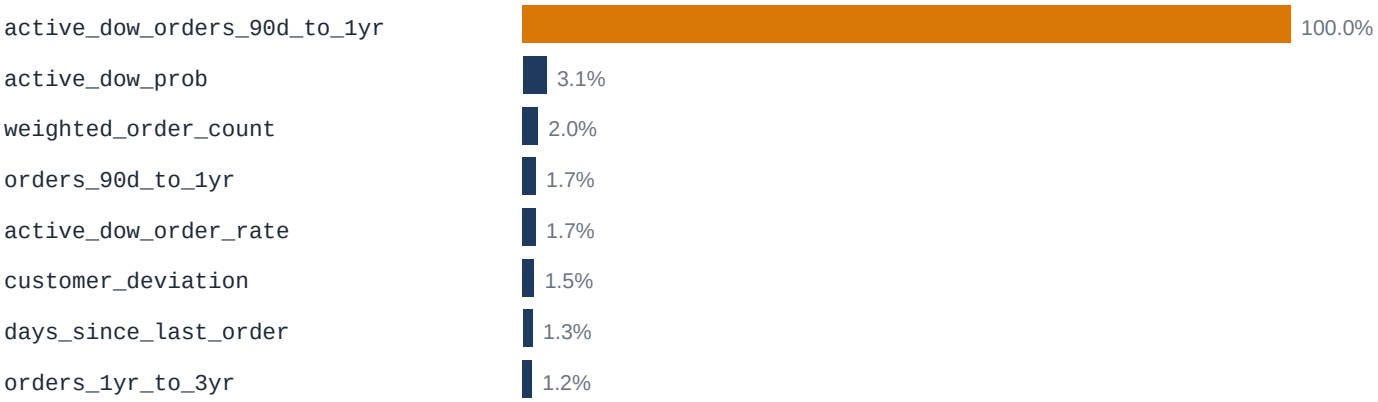
Feature engineering: 53 features per customer × product

Features fall into seven groups, each addressing a specific failure mode the early model versions exhibited:

Group	Features	What it captures
Recency weighting	orders_90d · orders_90d_to_1yr · orders_1yr_to_3yr · weighted_order_count	Recent orders weighted 3× over older ones: recency without throwing away history.
Quantity statistics	avg_qty_90d · avg_qty_all · max_qty_ever	Both typical and ceiling: the quantity model needs to know the customer's max.
DOW probabilities	dow_prob_{mon..sun}	Empirical frequency of each weekday for this customer × product.
DOW quantities	avg_qty_dow_{mon..sun}	Average quantity given the day. Monday loads differ from Thursday top-ups.
DOW recency	dow_orders_90d_{mon..sun} · dow_orders_90d_to_1yr_{mon..sun} · days_since_last_dow_{mon..sun}	How recently the customer ordered on this specific delivery day. Critical.
Active-DOW	active_dow_prob · active_dow_orders_90d_to_1yr · active_dow_order_rate · days_overdue	Pre-selected for the target cart's delivery day. Eliminates the need for the model to learn 7-way column selection.
Seasonal context	seasonal_multiplier · customer_deviation · trend_ratio	Region-aware STL × per-customer deviation × short-term trend.

Top features by gain · basket predictor

Day-of-week-specific recency dominates · classic recency is secondary



What the gain chart actually says

The dominant signal isn't classic recency. It's **day-of-week-specific recency**: how often the customer has ordered this item from you on this specific delivery day in the 90-day-to-1-year window. Classic recency ranks seventh. The reason matters: competing distributors in relationship-driven categories carry largely overlapping SKU coverage, so a customer's *product* preferences don't lock them to any one vendor. What's stable is the **purchasing slot**, like a Monday-from-you rhythm or a Thursday-from-someone-else rhythm, set by purchasing cadence rather than preference. The model is learning which slots you own. A confident prediction the customer dismisses means the customer's behavior diverged from their doppelgänger, and that's information worth investigating. The same change that surfaced this insight (per-row selection from 7-column DOW arrays) moved the positive rate from a contaminated 34% to a clean 12%, the real-world base rate for surviving competitive pressure on any given SKU.

STL seasonality with a 4-level fallback

Why region-aware seasonal decomposition matters, and how the system handles thin data.

Seasonality in distribution is regionally specific in ways that break naive global models. A product that peaks in summer in one region may peak in winter in another; categories tied to weather, school calendars, construction cycles, or holiday volume all decompose differently across geographies. The system fits an STL decomposition (`robust=True, period=52, seasonal=53`) per product × region, with holiday weeks 51/52/1 interpolated through before fitting to prevent New Year demand spikes from distorting the seasonal shape.

When a product × region combination doesn't have ≥78 weeks of clean history, lookup uses a four-level hierarchy at query time:

Level	Lookup	Used when
1	product_id × profit_center	Highest specificity available
2	product_id × region	No profit-center history
3	product_id × warehouse	No regional history
4	product_id × national aggregate (region=NULL)	Everything else

An earlier version used K-means clustering on weekly seasonal profiles to give thin-data products an archetype-based multiplier. Removed in v2. The fallback hierarchy handled the same edge cases more transparently, and stakeholders could explain a regional-then-national fallback in a meeting; nobody could explain a K-means archetype. Operational simplicity won.

STL parameter choices

- **period=52, seasonal=53.** The period is weekly seasonality over a year. STL requires seasonal to be odd and greater than period; 53 is the smallest valid value, which gives the tightest seasonal estimate without overfitting noise.
- **robust=True.** Iterative reweighting downweights outliers (one-off bulk orders, data errors, holiday anomalies) so they don't distort the seasonal shape. The cost is ~3× the compute; the benefit is signal that survives data quality issues.
- **Holiday weeks 51/52/1 interpolated.** Christmas and New Year crater B2B ordering volumes in ways that aren't seasonal. They're calendar artifacts. Interpolating these weeks through before STL prevents the decomposition from learning a fake December dip.
- **MIN_WEEKS_FOR_STL = 78.** The product needs 1.5 full annual cycles minimum. Below that threshold, the seasonal component is statistically untrustworthy and the system falls back one hierarchy level instead of fitting noisy curves.

Honest about thin data

With less than two years of weekly data, STL multipliers are flat at 1.0. This is correct behavior, not a bug. The feature matures as data accumulates and self-activates past the 78-week threshold per product × region. The regional multiplier captures what a category *should* do in a given region at a given time of year; a per-customer `customer_deviation` feature layers on top to capture each customer's historical departure from their region's pattern. Final multiplier: `regional_seasonal × customer_deviation`.

Training discipline & operational maturity

What separates this from a Kaggle-grade notebook is the surface area that has nothing to do with model accuracy.

Training discipline

- **Time-based holdout, not group-shuffle.** An earlier version used `GroupShuffleSplit` with customer as the group. That prevented customer leakage but allowed *temporal* leakage; future weeks bled into the training set. The current version splits by `week_start` with a 90-day validation window, which is the right shape for a system that runs against a moving present.
- **Sample weighting for recency.** Training rows are weighted to give the last 90 days 3× the influence of older data, mirroring the feature pipeline's recency weighting so model and features stay aligned.
- **Class imbalance handled at training, not inference.** The basket model uses `scale_pos_weight = n_neg / n_pos` on the training fold (~7:1 at the 12% real positive rate). This keeps the score distribution centered around 0.5, so the trigger engine's threshold range (0.40–0.80) stays meaningful without per-customer recalibration.

Operational maturity beyond the model

Each module below is what separates a Kaggle-grade notebook from a system that survives deployment. Most exist specifically to protect AE credibility and to let the doppelgänger mature properly before any of its predictions touch a customer. These are the two constraints that matter most in a relationship-driven industry.

Module	What it does
Silent learning mode	Trigger engine generates and scores carts for 4–6 weeks before any customer-facing notification. The point isn't model validation. It's letting the doppelgänger form before the AE has to stand behind its predictions. Telling a customer "the AI thinks you forgot X" when they bought X from a competitor yesterday is the kind of credibility hit that's hard to walk back.
Threshold optimizer	Per-customer threshold tuning with time-ordered train/validate splits and overfit-gap guardrails. Sweeps probabilities against the silent log to find the operating point that maximizes *operational* outcomes (acceptance rate, override rate, OOS recovery), not F1.
Model health monitor	Daily check on score distribution drift, feature-null rates, and acceptance trend. A sudden drop in acceptance is the leading indicator that account behavior is drifting from the doppelgänger. The monitor flags it before the case-count report would, and gives the AE a reason to make a relationship call.
OOS demand logger	When a substitute is shown and dismissed, the original OOS item is logged. An OOS dismissal in relationship-driven distribution isn't lost demand; it's demand that walked across the street. The weekly report quantifies recovery in dollars.
Accuracy tracker	Per-customer, per-product acceptance and quantity-delta tracking. Customers whose actual ordering behavior diverges meaningfully from their doppelgänger surface as relationship-conversation candidates. The AE gets the alert with weeks of runway.
Distributor onboarding	Multi-tenant from day one. New distributor: configure DB, choose region preset, run onboard script, load data, run pipelines.
EDI bridge	X12 850 (PO out), 846 (inventory in), 832 (catalog in). Pluggable provider abstraction for SPS Commerce, TrueCommerce, GXS, plus a dev stub.

The intelligence layer: Claude as customer-facing surface

Three distinct AI surfaces, one critic-validator pattern, zero hallucinated quantities.

Raw ML output isn't customer-facing. Three Claude-powered surfaces (model: `claude-sonnet-4-6`) make the system explainable and actionable. Each addresses a specific operator or customer need, and each runs through the same critic-validator pattern that prevents hallucinated quantities and product names from reaching customers.

1. Cart summary, streaming and customer-facing

When a customer opens their pending cart, a streaming natural-language summary explains why the cart was generated and calls out notable items, like seasonal increases, substitutions, frequently ordered products. The response streams token-by-token.

Critic-validator pattern

Every summary goes through a two-pass system. Claude generates it; a second Claude call fact-checks it against the actual cart data, verifying quantities, item names, and substitution claims. If errors are found, the corrected version is returned instead. This prevents hallucination of specific quantities or product details in customer-facing copy. A non-negotiable requirement when you're telling a customer they ordered 4 cases of something they've never bought.

2. Customer insight card, admin dashboard

Each customer detail page shows an AI-generated health summary for the distributor admin and the AE. Claude ingests structured signals (acceptance rate, days since last order, trigger log, consecutive dismissals) and produces a headline, narrative, signal chips, and recommended action. The same critic-validator pass checks that acceptance rate labels match the actual percentages and dismissal counts are accurate. Results are cached for one hour via Next.js `unstable_cache`.

Implementation gotcha worth documenting

The Supabase client must be created *inside* the cached function, not passed as an argument. `unstable_cache` serializes its cache-key arguments; a Supabase client has circular references that crash JSON serialization. The error surfaces as a vague serialization failure deep in Next.js internals. Took time to diagnose, and worth documenting for the next person.

3. Smart alerts, admin dashboard

Algorithmic anomaly detection across three checks, with no Claude needed for detection, only for summarization:

- **Missed order day:** customer scheduled today with no cart generated.
- **Overdue:** days since last order exceeds 2× typical interval. Requires `orders_90d ≥ 3` to be statistically valid; skips new accounts with insufficient history.
- **Consecutive dismissals:** last 3 carts all dismissed. Requires ≥3 total draft orders. The strongest early-warning signal that account behavior is drifting from the doppelgänger.

When two or more alerts exist, Claude generates a two-sentence fleet health summary. Individual alerts are always shown regardless of whether the summary call succeeds. The AI layer is additive, not load-bearing.

Engineering decisions worth noting

Five architectural decisions that shaped the system, each with the reasoning preserved so it can be defended in technical depth.

- **Global models, not per-customer.** Both LightGBM models are trained on all customers' data combined. At sub-100 customers, per-customer models overfit badly. The global model generalizes across behavior patterns and improves as the customer base grows; designed to scale to thousands of customers without architectural changes.
- **No cooldown policy on carts.** Early versions had cooldown logic that suppressed legitimate daily orders for high-frequency customers. Removed permanently. Carts are generated on every configured order day, period. Distribution customers order on rhythms the model should respect, not throttle.
- **Async Server Components + Suspense for the AI panels.** The Smart Alerts and Customer Insight Card are async React Server Components that fetch their own data. Wrapped in `<Suspense>`, they stream in independently without blocking the page render. The dashboard KPI cards load instantly; the AI panels arrive when ready.
- **Fly.io over Railway for the Python backend.** Supabase's free tier provides an IPv6-only database connection. Railway doesn't support outbound IPv6. Fly.io does. The connection works directly with no pooling workarounds. Infrastructure decisions aren't always about features; sometimes they're about which provider can reach your database.
- **Critic-validator on every customer-facing AI surface.** A second Claude pass validates quantities and product names before any AI-generated text reaches a customer. The cost is one extra API call; the benefit is never having to explain a hallucinated quantity to a customer who almost reordered something they've never bought.

Deployment stack

Layer	Technology
ML models	Python · LightGBM · scikit-learn · pandas · statsmodels
Backend API	FastAPI · SQLAlchemy · psycopg2
Frontend	Next.js 16 · TypeScript · Tailwind CSS v4 · shadcn/ui
Database	Supabase (PostgreSQL 17) · Row-Level Security
Auth	Supabase Auth · PKCE magic links for customer invites
AI	Anthropic Claude claude-sonnet-4-6 · streaming + JSON mode
Frontend hosting	Vercel
Backend hosting	Fly.io · IPv6-capable, connects to Supabase directly
Observability	Sentry · Playwright E2E · Lighthouse CI · Percy visual regression

What I'd build next

Five highest-leverage improvements. The first one productizes the doppelgänger-divergence signal the current system already implicitly surfaces.

- **Explicit divergence scoring against the doppelgänger.** The current architecture implicitly surfaces account drift through the silent log and acceptance tracker, but it doesn't productize the signal. A dedicated *behavioral divergence* output, combining declining acceptance trend, declining basket size, declining DOW-route consistency, and OOS dismissal velocity into a single score per customer, would give AEs a purpose-built dashboard for relationship attention. Independent operators don't churn loudly; they drift, one SKU at a time, and a productized divergence score catches that pattern early enough that the AE can have a real conversation, not a save attempt.
- **Calibrated probability outputs.** LightGBM's raw probability scores aren't well-calibrated for production threshold tuning. A Platt-scaling or isotonic-regression post-processing step would let the threshold optimizer reason about expected acceptance directly. Critical for the divergence score, which needs probabilities that mean what they say.
- **Per-DOW and per-category threshold granularity.** The threshold optimizer already tunes per-customer thresholds with overfit-gap guardrails. Every customer has their own confidence threshold. The next granularity step is splitting that threshold by delivery DOW and product category. A customer's high-volume Monday warrants a different operating point than their sparse Friday top-up; fast-moving consumables (urgent, perishable) warrant a different threshold than shelf-stable inventory (forgiving, batchable). Same optimizer machinery, narrower scopes.
- **Quantity uncertainty intervals.** The L1 regressor emits a point estimate. A quantile-regression variant (LightGBM supports `objective="quantile"`) would emit P50/P75/P90 estimates, letting the UI surface "order 4–6 cases" instead of "order 5." Operators trust ranges more than precise predictions they can't verify.
- **Demand-spike detection.** The `trend_ratio` feature catches gradual shifts. A change-point detector (Bayesian online change-point, for instance) would catch step-changes earlier (a customer expansion, a permanent location closure, a seasonal operational pivot) and trigger feature recomputation outside the standard weekly cadence.

Honest framing

This is a personal project, built outside any employer, on my own time, with no proprietary data. Training and validation use a dataset whose *shape* reflects observed patterns in relationship-driven distribution (order cadence by independent-account class, DOW distribution across purchasing rhythms, 10–12% basket sparsity, fat-tailed quantities with bulk-order outliers, regional seasonal differences) but the specific rows are synthetic. No customer identifiers, no real SKU mappings, no pricing data. The architecture is designed to deploy per-distributor with isolated databases. Distributors compete with each other and won't co-mingle order data.

I architected the system end-to-end, made all technical decisions, and used AI-assisted development (Claude) to accelerate implementation. Every architectural choice in this document is a deliberate decision I made and can defend in technical depth. Code is private; the full repository is available under NDA for serious discussions.

About the author

Andrew McCombs is a sales and analytics professional with deep domain expertise in relationship-driven B2B distribution. His work focuses on the intersection of operational domain knowledge and applied machine learning. MS in Data Analytics from Penn State (3.95 GPA); professional doctorate in progress. Bilingual in English and Spanish. Co-founder of MenuPulse (operations technology) and RedVeil Technologies (commercial penetration testing).

Andrew McCombs · Greater Phoenix Area · linkedin.com/in/andrew-mccombs-585512107

System Architecture

End-to-end data flow from raw orders to AI-explained pre-filled carts